

Unit-III

Greedy Method

- Greedy algorithm obtains an **optimal** solution by making a **sequence** of decisions.
- Decisions are made **one by one** in some order.
- Each decision is made using a **greedy-choice property** or greedy criterion.
- A decision, once made, is (usually) **not changed** later.

- It gives an optimal solution when applied to problems with the **greedy-choice** property.
- A **feasible solution** is a solution that satisfies the constraints.
- An **optimal solution** is a feasible solution that optimizes the **objective** function.

Greedy method control abstraction/ general method

Algorithm Greedy(a,n)

// a[1:n] contains the n inputs

{

 solution=0 ; //Initialize solution

for i=1 to n **do**

 {

 x:=Select(a);

if Feasible(solution , x) **then**

 solution=Union(solution , x)

else

 reject();

 }

return solution;

}

Fractional Knapsack Problem

- Given: A set S of n items, with each item i having
 - p_i - a positive profit
 - w_i - a positive weight
- Goal: Choose items, allowing fractional amounts(x_i), to maximize total profit but with weight at most m .

$$\text{maximize } \sum p_i x_i$$

$$1 \leq i \leq n$$

$$\text{subjected to } \sum w_i x_i \leq m$$

$$1 \leq i \leq n$$

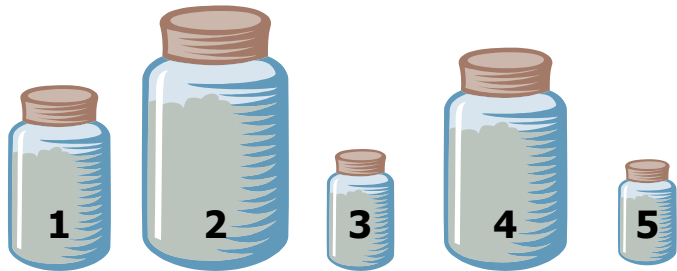
$$\text{and } 0 \leq x_i \leq 1, \quad 1 \leq i \leq n$$

Fractional Knapsack Problem

Greedy decision property:-

Select items in **decreasing** order of **profit/weight**.

Items:



w_i : 4 ml 8 ml 2 ml 6 ml 1 ml

p_i : \$12 \$32 \$40 \$30 \$50

Value:
(\$ per ml) 3 4 20 5 50



10 ml

"knapsack"

Solution:

- 1 ml of i_5
- 2 ml of i_3
- 6 ml of i_4
- 1 ml of i_2

- Solution vector

$$(x_1, x_2, x_3, x_4, x_5) = (0, 1/8, 1, 1, 1)$$

- Profit $= 12*0 + 32*1/8 + 40*1 + 30*1 + 50*1$
 $= 0+4+40+30+50$
 $= 124.$

Greedy algorithm for the fractional Knapsack problem

Algorithm GreedyKnapsack(m,n)

//P[1:n] and w[1:n] contain the profits and weights

// respectively of the n objects ordered such that

// $p[i]/w[i] \geq p[i+1]/w[i+1]$.

//m is the knapsack size and x[1:n] is the solution Vector.

{

 for i=1 to n do x[i]=0; // Initialize x.

 U=m;

 for i=1 to n do

 {

 if (w[i]>U) then break;

 if x[i]=1; U=U-w[i];

 }

 if (i <=n) then x[i]= U/w[i];

}

If you do not consider the time to sort the items, then the time taken by the above algorithm is **$O(n)$** .

0/1 Knapsack Problem

- An item is either included or not included into the knapsack.

Formally the problem can be stated as

$$\text{maximize } \sum p_i x_i$$

$$1 \leq i \leq n$$

$$\text{subjected to } \sum w_i x_i \leq m$$

$$1 \leq i \leq n$$

$$\text{and } x_i = 0 \text{ or } 1, \quad 1 \leq i \leq n$$

Which items should be chosen to **maximize** the amount of **money** while still keeping the overall weight under m kg?

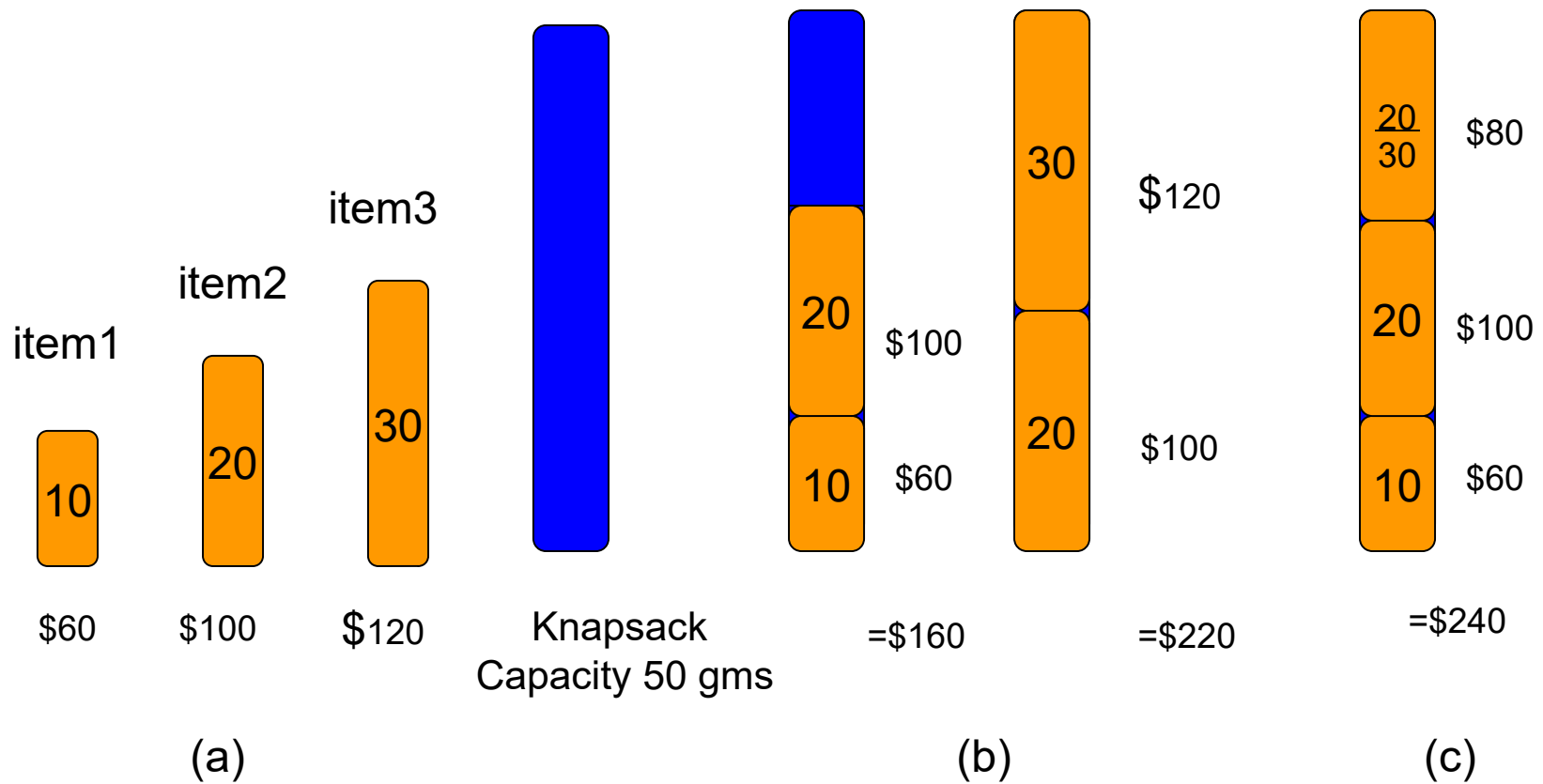


m



Is the fractional knapsack algorithm applicable?

- The greedy method works for **fractional** knapsack problem, but it does not for **0/1** knapsack problem.
- Ex:-



- There are 3 items, the knapsack can hold **50** gms.
- The value per gram of item 1 is 6, which is greater than the value per gram of either item2 or item3.
- The *greedy approach* (Decreasing order of profit's/weight's), does not give an optimal solution.
- As we can see from the above fig., the *optimal solution* takes item2 and item3.
- For the *fractional* problem, the *greedy approach* (Decreasing order of profit's/weight's) gives an *optimal solution* as shown in fig c.

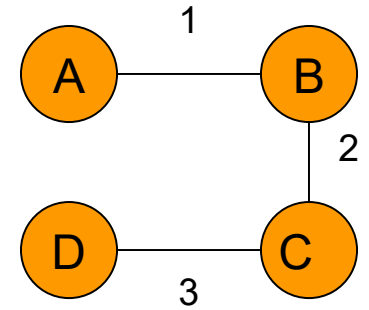
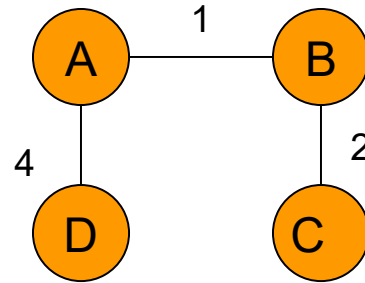
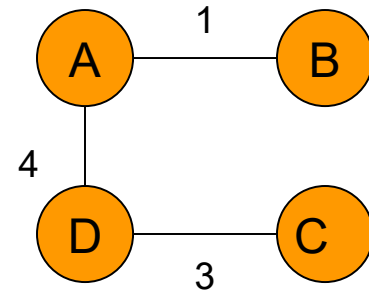
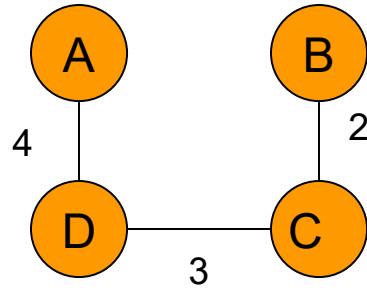
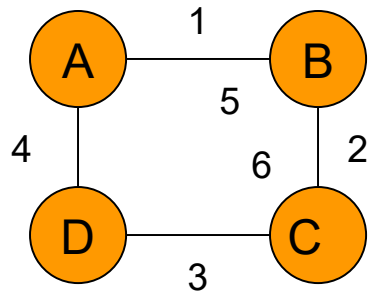
Spanning Tree

- A *tree* is a connected undirected graph that contains **no cycles**.
- A *spanning tree* of a graph G is a subgraph of G that is a *tree* and contains **all** the vertices of G .

Properties of a Spanning Tree

- The spanning tree of a n -vertex undirected graph has exactly $n - 1$ edges.
- It connects all the vertices in the graph.
- A spanning tree has no cycles.

Ex:-



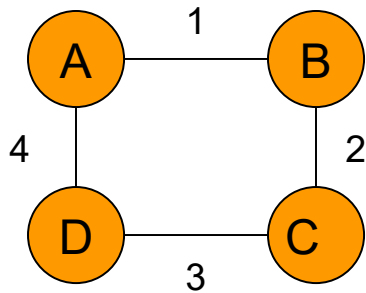
...

Some Spanning Trees

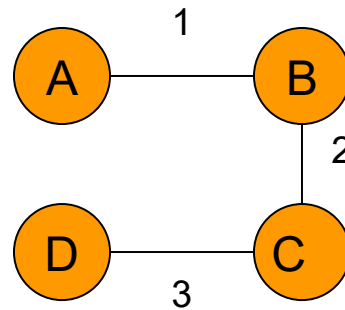
Undirected Graph

Minimum Cost Spanning Tree / Minimum Spanning Tree (MST)

- A *minimum spanning tree* is the one among all the spanning trees with the *smallest total cost*.



Undirected Graph



Minimum Spanning Tree

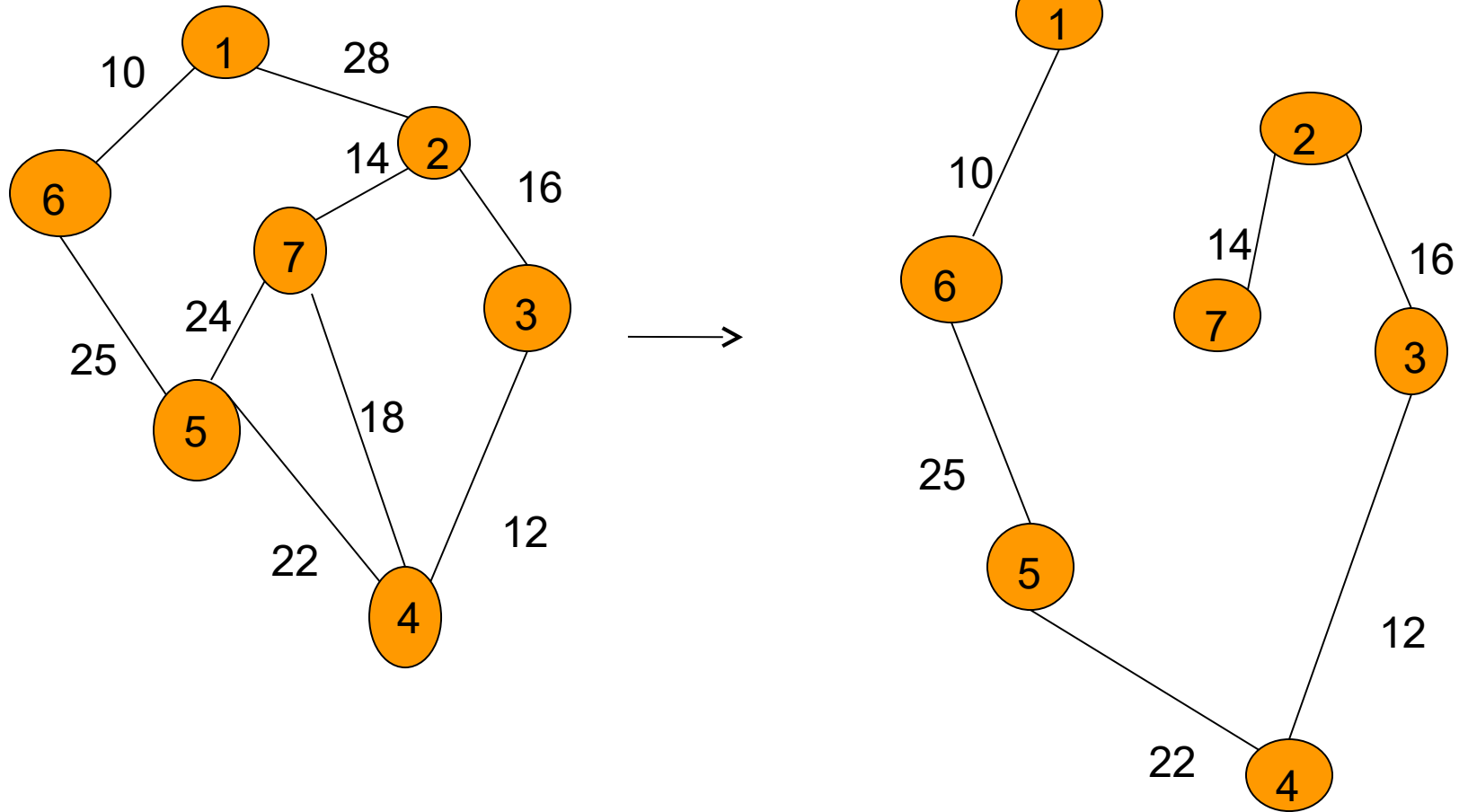
Applications of MSTs

- Computer Networks
 - To find how to connect a set of computers using the minimum amount of wire.

MST-Prim's Algorithm

- Start with minimum cost edge.
- Select next minimum cost edge (i, j) such that i is a vertex already included in the tree, j is a vertex not yet included.
- Continue this process until the tree has $n - 1$ edges.

Prim's Algorithm



Algorithm:

```
Algorithm prim(adjmatrix[1..n],[1..n])
{
//n=no. of nodes in the graph
//nEdges=current no. of edges included in the MST
//min=minimum weight(cost) of edge
//row,col indicate the row & col of the edge included
in the MST
//sum=total min weight cost of the edges included in
the MST
//initialize selected[ ] with zeros
```

```
Selected[1]=1; //to start with node 1 included in MST
nEdges=1;
sum=0;
while(nEdges<n)
min=infinity;
for i=1 to n do
    {
        if (selected[i]=1) //if node I is included in
                               MST then repeat to get the
                               min cost node j to connect to i
        for j=2 to n do //select the min cost edge
            {
```

```
If (selected[j]=0) //select the min cost edge
    if (min>adjmatrix[i][j])
        min=adjmatrix[i][j];
print adjmatrix[i][j]);
}
selected[j]=1; //once i & j form a min edge mark 1
as selected
nEdges++;
sum=sum+ adjmatrix[i][j];
}
print sum;
}
```

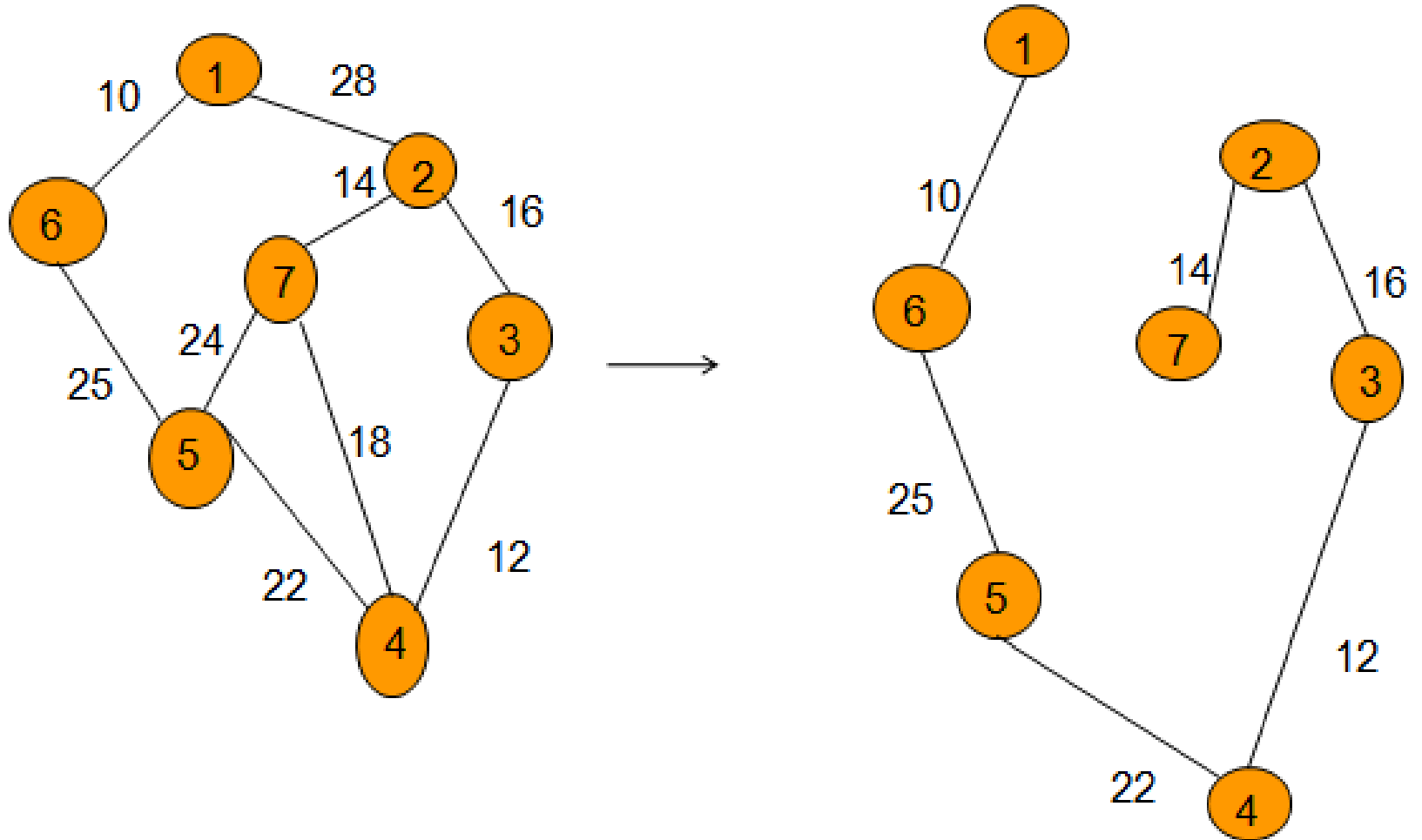
Best Case: The time complexity of Prim's Algorithm can be reduced to $O(E \log V)$ using an adjacency list.

Worst Case: But in a complete graph, $E = V * (V - 1) = (V^2)$.

Kruskal's Method

- Start with a forest that has no edges.
- Add the next minimum cost edge to the forest if it will not cause a cycle.
- Continue this process until the tree has $n - 1$ edges.

Kruskal's Algorithm



Algorithm Kruskals(E, cost, n, t)

{

i=0;

mincost=0;

while(i<=n-1)

{

 delete a min cost edge(u,v);

 j=find(u);

 k=find(v);

 if(j! = k)

 {

 i=i+1;

```
t[i:1]=u;
```

```
t[i:2]=v;
```

```
Mincost=mincost + cost(u,v);
```

```
union (j,k);
```

```
}
```

```
}
```

In Kruskal's algorithm, we first sort all graph edges by their weights. This operation takes $O(E \log E)$ time, where E is the total number of edges. Then we use a loop to go through the sorted edge list. In each iteration, we check whether a cycle will be formed by adding the edge into the current spanning tree edge set.

Disjoint Sets

➤ Two sets A and B are said to be **disjoint** if there are no common elements i.e., $A \cap B = \emptyset$.

➤ *Example:*

1) $S_1 = \{1, 7, 8, 9\}$, $S_2 = \{2, 5, 10\}$, and $S_3 = \{3, 4, 6\}$.
are three disjoint sets.

- We identify a set by choosing a *representative element* of the set. It doesn't matter which element we choose, but once chosen, it can't be changed.
- Disjoint set operations:
 - FIND-SET(x): Returns the representative of the set containing x .
 - UNION(i, j): Combines the two sets i and j into one new set. A new representative is selected.
(Here i and j are the representatives of the sets)

Disjoint Sets Example

- Make-Set(1)
- Make-Set(2)
- Make-Set(3)
- Make-Set(4)
- Union(1,2)
- Union(3,4)
- Find-Set(2) returns 1
- Find-Set(4) returns 3
- Union(2,4)



MST-Kruskal's Algorithm

Algorithm kruskal(E, cost, n, t)

// E is the set of edges in G .

// $\text{cost}[1:n, 1:n]$ is the cost matrix such that $\text{cost}[i, j]$ is either

// positive real number or ∞ if no edge (i, j) exists. $\text{cost}[i, j] = 0$, if $i = j$.

// A minimum spanning tree is computed and stored

// as a set of edges in the array $t[1:n-1]$.

{

 for $i := 1$ to n do

Make-Set(i); // each vertex is in a different set.

Sort the edges of E into increasing order by cost.

```

    mincost:=0; i:=1;
    for each edge (u,v)  $\in$  E, taken in increasing order by cost
    do
        {   j:= Find-Set(u); k:= Find-Set(v)
            if( j $\neq$ k ) then
                {
t[i,1]:=u; t[i,2]:=v;
mincost:=mincost+cost[u,v];
Union(j,k);
i:=i+1;
                }
        }
    return mincost;
}

```

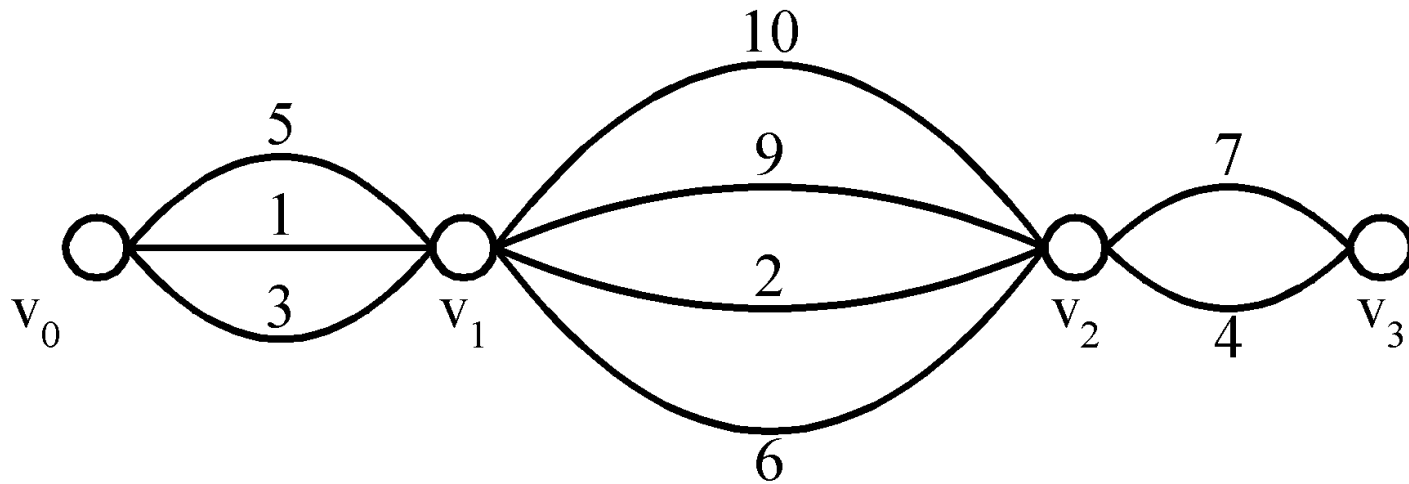

Time complexity of kruskal's algorithm

- With an efficient **Find-set** and **union** algorithms, the running time of kruskal's algorithm will be dominated by the time needed for sorting the edge costs of a given graph.
- Hence, with an efficient sorting algorithm(**merge sort**), the complexity of kruskal's algorithm is **$O(E \log E)$.**

Example:

Shortest Path on a Special Graph

- Problem
 - Find a **shortest** path from v_0 to v_3
- Greedy Solution



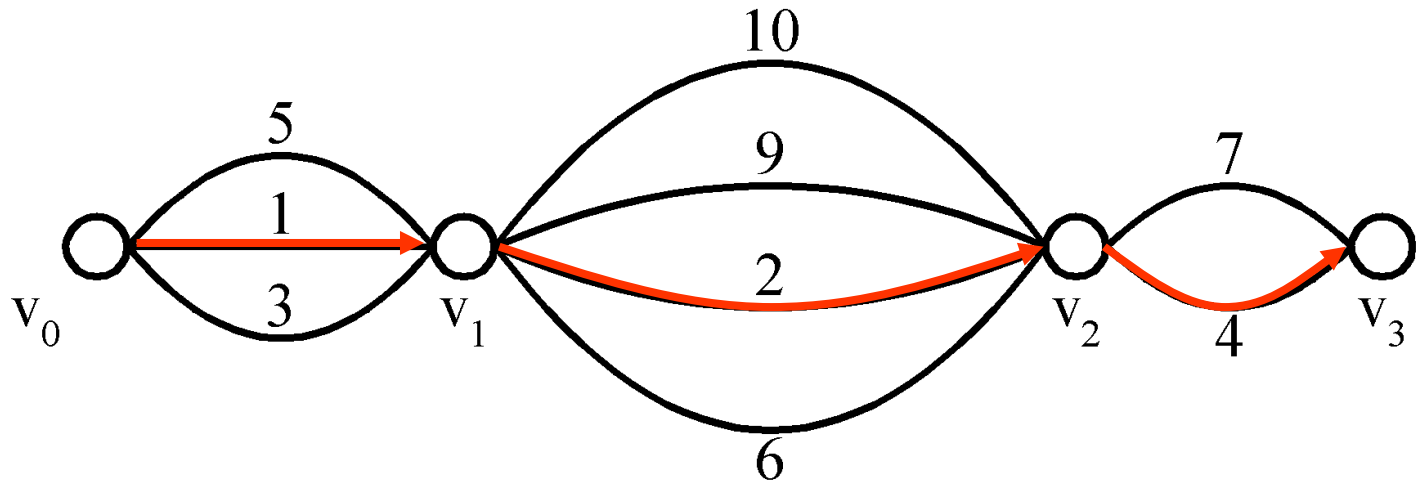
Example:

Shortest Paths on a Special Graph

- Problem
 - Find a **shortest** path from v_0 to v_3

- Greedy Solution

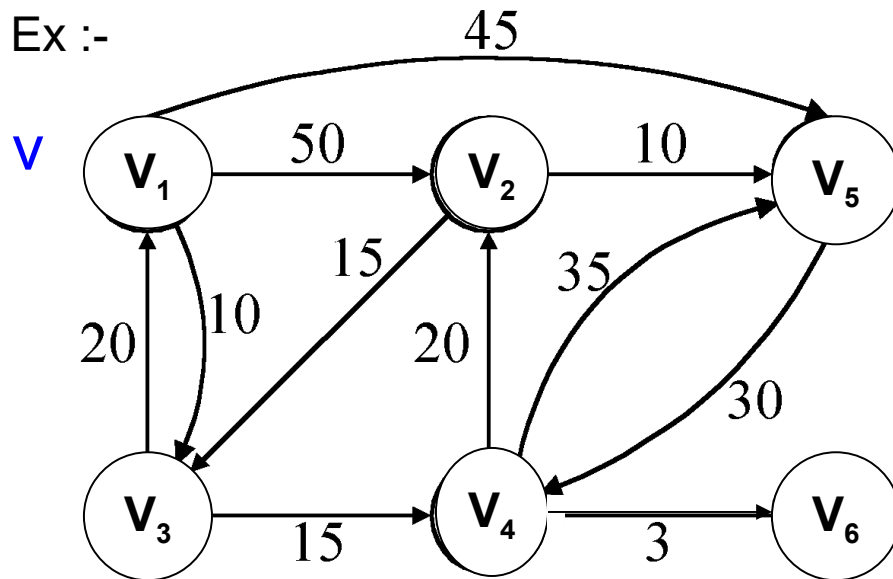
Is the solution optimal?



The Single-Source Shortest path Problem (SSSP)

- Given a **positively** weighted directed graph G with a source vertex v , find the shortest paths **from v to all other vertices** in the graph.

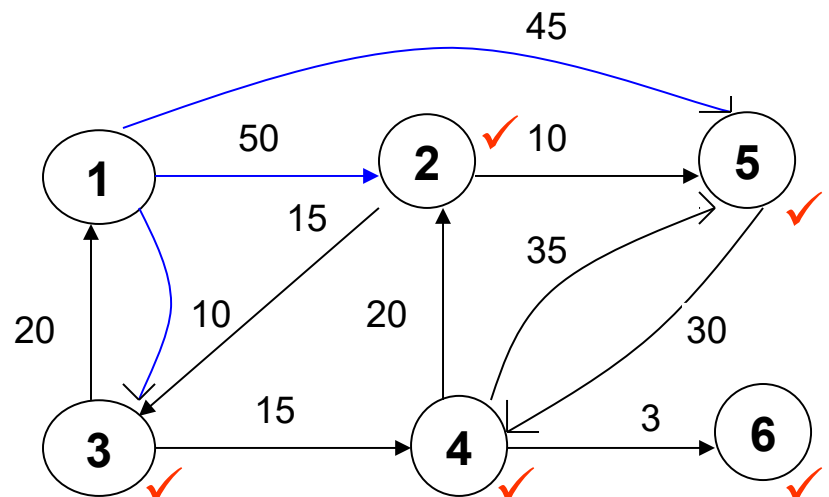
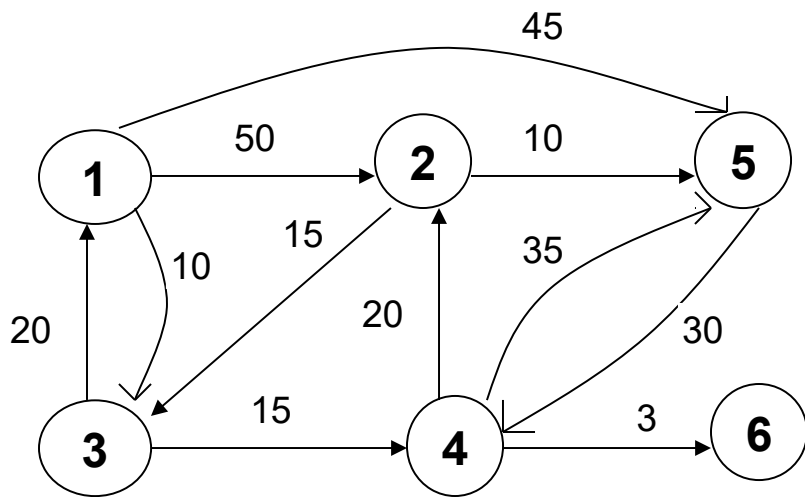
Ex :-



(a)

	<u>Path</u>	<u>Length</u>
1)	$v_1 v_3$	10
2)	$v_1 v_3 v_4$	25
3)	$v_1 v_3 v_4 v_2$	45
4)	$v_1 v_5$	45
5)	$v_1 v_3 v_4 v_6$	28

(b)



Iteration	S	Dist[2]	Dist[3]	Dist[4]	Dist[5]	Dist[6]
Initial	{1}	50	10 ✓	25	45	∞
1	{1,3}	45	10	25 ✓	45	∞
2	{1,3,4}	45	10	25	45	28 ✓
3	{1,3,4,6}	45	10	25	45 ✓	28
4	{1,3,4,5,6}	45 ✓	10	25	45	28

SSSP-Dijkstra's algorithm

- Dijkstra's algorithm assumes that $\text{cost}(e) \geq 0$ for each e in the graph.
- Maintains a set S of vertices whose SP from v (source) has been determined.
- a) Select the next minimum distance node u , which is not in S .
- (b) *for* each node w adjacent to u *do*
 - if($\text{dist}[w] > \text{dist}[u] + \text{cost}[u, w]$) then
 - $\text{dist}[w] := \text{dist}[u] + \text{cost}[u, w];$
- Repeat step (a) and (b) until $S = n$ (number of vertices).

```

1 Algorithm ShortestPaths(v, cost, dist, n)
2 //dist[s],  $1 \leq j \leq n$ , is the length of the shortest path
3 //from vertex v to vertex j in a digraph G with n vertices.
4 // dist[v] is set to zero. G is represented by its cost
  adjacency
5 // matrix cost[1:n, 1:n].
6 {
7   for i:= 1 to n do
8     { // Initialize s.
9       s[i]:=false; dist[i]:=cost[v,i];
10      }
11 s[v]:=true; // put v in S.
12 dist[v]= 0.0;

```

```
13  for num:=2 to n-1 do
14      {
15  Determine n-1 paths from v.
// Choose u from among those vertices not in S such that
    dist[u] is minimum;

16      s[u]:=true;  // Put u in S.
17      for ( each w adjacent to u with s[w]= false) do
18          // update distances
19          if( dist[w]>dist[u]+cost[u,w]) ) then
20              dist[w]:=dist[u]+cost[u,w];
21      }
22 }
```

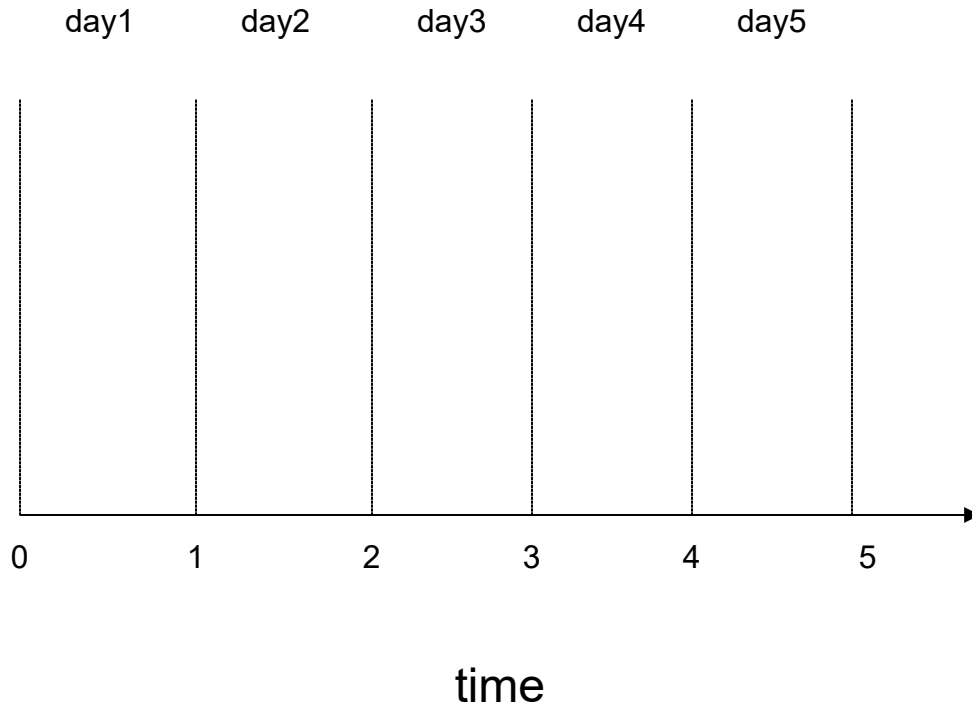

Time complexity of Dijkstra's Algorithm

- The for loop of line 7 takes $o(n)$.
- The for loop of line 13 takes $o(n)$.
 - Each execution of this loop requires $o(n)$ time
 - So the total time for this loop is $o(n^2)$.
- Therefore, **total time taken by this algorithm is $o(n^2)$.**

Job sequencing with deadlines

- We are given a set of n jobs.
- *Deadline* $d_i \geq 0$ and a *profit* $p_i > 0$ are associated with each *job* i .
- For any job profit is earned if and only if the job is *completed by its deadline*.
- To complete a job, a job has to be processed by a machine for *one unit* of time.
- Only *one machine* is available for processing jobs.
- A *feasible solution* of this problem is a *subset of jobs* such that each job in this subset can be completed by its deadline
- The *optimal solution* is a feasible solution which will *maximize* the total profit.
- The *objective* is to find an *order of processing of jobs* which will *maximize the total profit*.

Ex:- $n=4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$
 $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$



Ex:- $n=4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$ $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

The maximum deadline is 2 units, hence the feasible solution set must have ≤ 2 jobs.

	feasible solution	processing sequence	value
1.	(1,2)	2,1	110
2.	(1,3)	1,3 or 3,1	115
3.	(1,4)	4,1	127
4.	(2,3)	2,3	25
5.	(3,4)	4,3	42
6.	(1)	1	100
7.	(2)	2	10
8.	(3)	3	15
9.	(4)	4	27

Solution 3 is optimal.

Greedy Algorithm for job sequencing with deadlines

1. Sort p_i into decreasing order. After sorting
$$p_1 \geq p_2 \geq p_3 \geq \dots \geq p_i.$$
2. Add the next job i to the solution set if i can be completed by its deadline.
3. Stop if all jobs are examined. Otherwise, go to step 2.

Ex:- 1) $n=5$, $(p_1, \dots, p_5) = (20, 15, 10, 5, 1)$ and
 $(d_1, \dots, d_5) = (2, 2, 1, 3, 3)$

The optimal solution is $\{1, 2, 4\}$ with a profit of 40.

Ex:- 2) $n=7$, $(p_1, \dots, p_7) = (3, 5, 20, 18, 1, 6, 30)$ and
 $(d_1, \dots, d_7) = (1, 3, 4, 3, 2, 1, 2)$

Find out an optimal solution.

```
Algorithm   JobSeq(J[1..n],P[1..n],d[1..n],list[1..n],n)
{
Initialize list with zeros //make all n time slots empty
Profit=0;
for i=1 to n do
{
    k=d[i]; //set ith job's deadline to k
    while(k>0)
    {
        if(list[k]=0) //if kth slot is vacant
            list[k]=job[i]; //assign ith job to kth slot
```

```
profit=profit + p[i]; //accumulate total profit
    goto step(for loop);
}
    k=k-1; //otherwise get preceeding slot
}
}
Print list
}
```

Time taken by this algorithm is $O(n^2)$